

Java gRPC Bidirectional Streaming

This tutorial provides a basic Java programmer's introduction to working with gRPC streaming.

By the end of the lab you should be able to:

- define a streaming RPC in a .proto file
- generate Java gRPC classes
- implement StreamObserver on the server and client and run a bidirectional streaming application in NetBeans.

gRPC Service and streaming

Like many RPC systems, gRPC is based around the idea of defining a service, specifying the methods that can be called remotely with their parameters and return types. By default, gRPC uses protocol buffers as the Interface Definition Language (IDL) for describing both the service interface and the structure of the payload messages. It is possible to use other alternatives if desired.

gRPC lets you define four kinds of service methods:

- Unary RPCs where the client sends a single request to the server and gets a single response back, just like a normal function call.

rpc SayHello(HelloRequest) **returns** (HelloResponse);

- Server streaming RPCs where the client sends a request to the server and gets a stream to read a sequence of messages back. The client reads from the returned stream until there are no more messages. gRPC guarantees message ordering within an individual RPC call.

rpc LotsOfReplies(HelloRequest) **returns** (stream HelloResponse);

- Client streaming RPCs where the client writes a sequence of messages and sends them to the server, again using a provided stream. Once the client has finished writing the messages, it waits for the server to read them and return its response. Again gRPC guarantees message ordering within an individual RPC call.

rpc LotsOfGreetings(stream HelloRequest) **returns** (HelloResponse);

- Bidirectional streaming RPCs where both sides send a sequence of messages using a read-write stream. The two streams operate independently, so clients and servers can read and write in whatever order they like: for example, the server could wait to receive all the client messages before writing its responses, or it could alternately read a message then write a message, or some other combination of reads and writes. The order of messages in each stream is preserved.

rpc BidiHello(stream HelloRequest) **returns** (stream HelloResponse);

Exercise: Develop a Java gRPC client-server application that demonstrates bidirectional streaming. The client should generate 10,000 random integers every second for a total duration of 10 seconds, resulting in approximately 100,000 integers being produced. Each generated integer should be streamed to the gRPC server as it is created. The server must process each incoming number, determine whether it is odd or even, and immediately stream a response back to the client containing the original number and its classification as ODD or EVEN. The client should receive and display the responses returned by the server while the streaming process is still in progress.

1. Create a Maven Project in NetBeans:

1. Open NetBeans.
2. Select File > New Project.
3. Choose Java with Maven > Java Application.
4. Click Next.
5. Use Project Name: GrpcNumberStreaming.
6. Use Group Id: com.example.
7. Use Version: 1.0-SNAPSHOT.
8. Click Finish.

2. Add gRPC Dependencies:

1. Open pom.xml and replace its contents with the following Maven configuration. This configuration downloads gRPC, Protocol Buffers, and the code-generation plugin.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>GrpcNumberStreaming</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <grpc.version>1.84.0</grpc.version>
    <protobuf.version>3.25.8</protobuf.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-netty-shaded</artifactId>
      <version>${grpc.version}</version>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-protobuf</artifactId>
      <version>${grpc.version}</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-stub</artifactId>
      <version>${grpc.version}</version>
    </dependency>
  </dependencies>
```

```

        <groupId>javax.annotation</groupId>
        <artifactId>javax.annotation-api</artifactId>
        <version>1.3.2</version>
    </dependency>
</dependencies>

<build>
    <extensions>
        <extension>
            <groupId>kr.motd.maven</groupId>
            <artifactId>os-maven-plugin</artifactId>
            <version>1.7.1</version>
        </extension>
    </extensions>

    <plugins>
        <plugin>
            <groupId>org.xolstice.maven.plugins</groupId>
            <artifactId>protobuf-maven-plugin</artifactId>
            <version>0.6.1</version>
            <configuration>
                <protocArtifact>
                    com.google.protobuf:protoc:${protobuf.version}:exe:${os.detected.classifier}
                </protocArtifact>
                <pluginId>grpc-java</pluginId>
                <pluginArtifact>
                    io.grpc:protoc-gen-grpc-java:${grpc.version}:exe:${os.detected.classifier}
                </pluginArtifact>
            </configuration>
            <executions>
                <execution>
                    <goals>
                        <goal>compile</goal>
                        <goal>compile-custom</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
</project>

```

2. Save the pom.xml file. Check the NetBeans dependencies folder; it should automatically download the necessary dependencies.

3. Create the .proto File:

The standard Maven location for Protocol Buffer definitions is src/main/proto.

1. Create the folder src/main/proto under the project directory.
2. Inside src/main/proto create a new empty file named *number.proto*.
3. Add the following code to the number.proto file.

```

syntax = "proto3";

option java_package = "com.example.grpcstream";
option java_multiple_files = true;

service NumberService {
    rpc CheckNumbers(stream NumberRequest)
        returns (stream NumberResponse);
}

message NumberRequest {
    int32 number = 1;
}

```

```
message NumberResponse {
    int32 number = 1;
    string result = 2;
}
```

4. Generate the Java gRPC Classes:

1. Right-click the project in NetBeans.
2. Choose Clean and Build.
3. Maven compiles number.proto and generates Java classes.
4. If the build succeeds, generated classes include NumberRequest, NumberResponse, and NumberServiceGrpc.

5. Create the gRPC Server:

1. Under Source Packages, create the package com.example.grpcstream.
2. Right-click the package and choose New > Java Class.
3. Name the class NumberServer.
4. Implement the server code as follows:

```
package com.example.grpcstream;

import io.grpc.Server;
import io.grpc.ServerBuilder;
import io.grpc.stub.StreamObserver;
import java.io.IOException;
import java.util.concurrent.atomic.AtomicInteger;

public class NumberServer {

    public static void main(String[] args)
        throws IOException, InterruptedException {

        Server server = ServerBuilder
            .forPort(50051)
            .addService(new NumberServiceImpl())
            .build()
            .start();

        System.out.println("Number Server started on port 50051...");
        server.awaitTermination();
    }

    static class NumberServiceImpl
        extends NumberServiceGrpc.NumberServiceImplBase {

        private final AtomicInteger totalCount = new AtomicInteger();
        private final AtomicInteger oddCount = new AtomicInteger();
        private final AtomicInteger evenCount = new AtomicInteger();

        @Override
        public StreamObserver<NumberRequest> checkNumbers(
            StreamObserver<NumberResponse> responseObserver) {

            return new StreamObserver<NumberRequest>() {

                @Override
                public void onNext(NumberRequest request) {
                    int number = request.getNumber();
                    totalCount.incrementAndGet();

                    String result;
                    if (number % 2 == 0) {
```



```

        public void onNext(NumberResponse response) {
            int count = responsesReceived.incrementAndGet();

            // Display every 1000th response to avoid flooding the console.
            if (count % 1000 == 0) {
                System.out.println(
                    "Response " + count + ": "
                    + response.getNumber() + " -> "
                    + response.getResult());
            }
        }

        @Override
        public void onError(Throwable t) {
            System.err.println("Server error: " + t.getMessage());
            completed.countDown();
        }

        @Override
        public void onCompleted() {
            System.out.println("Server completed the response stream.");
            completed.countDown();
        }
    };

    StreamObserver<NumberRequest> requestObserver =
        stub.checkNumbers(responseObserver);

    Random random = new Random();
    int numbersSent = 0;
    long programStart = System.currentTimeMillis();

    for (int second = 1; second <= 10; second++) {

        long secondStart = System.currentTimeMillis();

        for (int i = 0; i < 10000; i++) {
            int number = random.nextInt(1_000_000) + 1;

            NumberRequest request = NumberRequest.newBuilder()
                .setNumber(number)
                .build();

            requestObserver.onNext(request);
            numbersSent++;
        }

        long elapsed = System.currentTimeMillis() - secondStart;
        long sleepTime = 1000 - elapsed;

        if (sleepTime > 0) {
            Thread.sleep(sleepTime);
        }

        System.out.println("Second " + second + " completed.");
    }

    requestObserver.onCompleted();

    completed.await(30, TimeUnit.SECONDS);

    long programEnd = System.currentTimeMillis();
    double executionTime = (programEnd - programStart) / 1000.0;

    System.out.println();
    System.out.println("----- CLIENT SUMMARY -----");
    System.out.println("Total numbers sent: " + numbersSent);
    System.out.println("Total responses received: "
        + responsesReceived.get());
    System.out.printf("Execution time: %.2f seconds%n",
        executionTime);

    channel.shutdown();
    channel.awaitTermination(5, TimeUnit.SECONDS);

```

```
}  
}
```

7. Run the Server and Client:

A. Start the server

1. Open `NumberServer.java`.
2. Right-click inside the editor and choose Run File, or press Shift+F6.
3. Leave the server running.

```
Number Server started on port 50051...
```

B. Start the client

1. Open `NumberClient.java`.
2. Right-click and choose Run File, or press Shift+F6.
3. Watch the Output window for ten seconds of streaming activity.

```
Response 1000: 64182 -> EVEN  
Second 1 completed.  
Response 10000: 739981 -> ODD  
Second 2 completed.  
...  
Second 10 completed.  
Server completed the response stream.  
  
----- CLIENT SUMMARY -----  
Total numbers sent: 100000  
Total responses received: 100000  
Execution time: 10.xx seconds
```

After the exercise, read the gRPC documentation and guides to help you with Assignment 2.